SimpleMultithreader

Overview

The SimpleMultithreader is a simple header-only implementation that parallelizes 1D and 2D loops using Pthreads. It divides work into chunks, creates threads, executes the loop body using C++11 lambda expressions, joins all threads, and prints total execution time. It follows all assignment rules and does not use any thread pools or C++ threading APIs.

Header Files Used

stdio.h – For printing output.
pthread.h – For creating and managing threads.
sys/time.h – For measuring execution time.
functional – For storing lambda functions.
vector – For storing thread IDs.
cstdlib – For exit handling.
string.h – For strerror() in error messages.

Functions

time_difference(struct timeval *start, struct timeval end)*
Computes the time difference in seconds.

thread_1d_loop_function(void arg)
Thread function for executing the assigned chunk of the 1D loop.

parallel_for(int low, int high, std::function<void(int)> &&lambda, int numThreads)
Parallelizes a 1D for-loop by dividing iterations into chunks and creating threads.

threadarg_1d_loop
Stores start index, end index, and lambda for 1D threads.

thread_2d_loop_function(void arg)
Thread function for executing assigned rows of the 2D loop.

parallel_for(int low1, int high1, int low2, int high2, std::function<void(int,int)> &&lambda, int numThreads)
Parallelizes a 2D nested loop by distributing rows across threads.

threadarg_2d_loop
Stores row range, column range, and lambda for 2D threads.

Link :
https://github.com/Swaraaj333/group-48

Work Division

1. Prerak Tanwar
   a) 1D parallel_for
   b) 1D thread function
   c) Chunk division logic

2. Swaraaj Krishna
   a) 2D parallel_for
   b) 2D thread function
   c) Time measurement and struct setup

AI Policy

I have understood and use AI for writing this part of the code

```
//this make worker threads for the remaining chunks

    for(int t = 1; t < numb_thread; t++){

        int this_chunk = chunks_siz;



        if(t < excess_chunk){

            this_chunk += 1;

        }



        //this makes the arguments

        threadarg_1d_loop* arg = new threadarg_1d_loop();

        arg->i_start = curr_start;

        arg->i_end   = curr_start + this_chunk;

        arg->lambda_fn = lambda;



        //moving to the next chunk start
```

```
//this make the worker thread for the remaining row blocks

   for(int t = 1; t < numb_thread; t++) {

       int this_chunk = chunks_siz;

       if(t < excess_chunk) {

           this_chunk += 1;

       }



       //this make the arg for 2d loop

       threadarg_2d_loop* arg = new threadarg_2d_loop();

       arg->row_i_start = curr_start;

       arg->row_i_end = curr_start + this_chunk;

       arg->col_j_start = low2;

       arg->col_j_end = high2;

       arg->lambda_fn = lambda;



       //this updates the next row start

       curr_start += this_chunk;
```